

UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO



FACULTAD DE INGENIERÍA

Laboratorio de Computación Gráfica e Interacción Humano-Computadora

Proyecto Final:

Animación y Modelado 3D de un Rancho

ALUMNOS:

JIMÉNEZ VÁZQUEZ SERGIO YAHIR — N. Cuenta: **320159204**

NEQUIS SARABIA JORGE LUIS — N. Cuenta: **320176993**

GRUPO: 04

PROFESOR: Ing. Eden Espinoza Urzua

SEMESTRE: 2027-1

FECHA DE ENTREGA: 09/11/2026

Índice

1 Introducción **2**

2 Manual de usuario **3**

2.1 Requisitos y ejecución . . . 3
2.2 Descripción del entorno . . . 3
2.3 Control de la cámara . . . 3
2.4 Resumen general de controles . . . 3
2.5 Iluminación . . . 4
2.6 Caballo . . . 4
2.7 Molino de viento . . . 4
2.8 Cochera . . . 4
2.9 Tractor . . . 4
2.10 Toro . . . 5
2.11 Abeja . . . 5
2.12 Sistema de animación por keyframes . . . 5
2.13 Solución de problemas . . . 6

3 Manual técnico y documentación del proyecto **7**

3.1 Objetivos . . . 7
3.2 Alcance del proyecto . . . 7
3.3 Limitantes . . . 8
3.4 Metodología de software aplicada . . . 8
3.5 Diagrama de flujo del software . . . 9
3.6 Diagrama de Gantt . . . 10
3.7 Inventario de objetos de la escena . . . 11
3.8 Documentación del código . . . 12

4 Conclusiones **18**

4.1 Nequis Sarabia Jorge Luis . . . 18
4.2 Jiménez Vázquez Sergio Yahir . . . 18

5 Referencias **19**

1 Introducción

El presente documento reúne el manual de usuario y el manual técnico del proyecto final **Animación y Modelado 3D de un Rancho**, desarrollado en C++ con OpenGL. El proyecto es un simulador interactivo en tiempo real: el usuario recorre libremente un rancho tridimensional con cámara en primera persona, controla varios de sus elementos con el teclado y puede grabar y reproducir animaciones por fotogramas clave (keyframes).

Algunos modelos se diseñaron en Blender, se exportaron en formato OBJ y se cargan en tiempo de ejecución mediante Assimp a través de la clase `Model` del código base proporcionado por el profesor (Práctica 10). Toda la animación se genera por código, sin animaciones pregrabadas dentro de los modelos.

El documento se organiza en dos partes: el **Manual de usuario**, con los controles y el modo de operación, y el **Manual técnico**, con objetivos, alcance, limitantes, metodología, diagrama de flujo, diagrama de Gantt, inventario de objetos y documentación detallada del código. El capítulo 4 presenta las conclusiones individuales de los integrantes del equipo.

2 Manual de usuario

2.1 Requisitos y ejecución

- Equipo con Windows y tarjeta gráfica con controlador compatible con OpenGL 3.3 o superior.
- El ejecutable debe conservar, junto a él, las carpetas **Models** (archivos .obj, .mtl y texturas) y **Shader** (lighting.vs, lighting.frag, lamp.vs, lamp.frag), porque el programa las busca con rutas relativas.
- Al iniciar se abre la ventana **Animacion Rancho** de 2024 x 1080 píxeles. Además se abre una consola donde el sistema imprime mensajes de estado de los keyframes.
- Para salir del programa se presiona **Esc**.

2.2 Descripción del entorno

El usuario aparece en el rancho en la posición (0, 2, 10), mirando hacia el centro de la escena, sobre un terreno de pasto delimitado por una cerca perimetral de madera. En el entorno se encuentran el establo (casita roja) con el caballo y la vaca en su interior, el puerquito, el pozo, la cochera azul con el tractor al frente, el molino de viento, el toro caminando por el terreno y una abeja que vuela en círculos sobre el centro del rancho. El cielo es de color azul claro.

2.3 Control de la cámara

La cámara es libre, en primera persona. Se desplaza con el teclado y se orienta con el ratón; no es necesario hacer clic.

Control	**Acción**
W	Avanzar en la dirección en que mira la cámara
S	Retroceder
A	Desplazarse lateralmente a la izquierda
D	Desplazarse lateralmente a la derecha
Ratón (movimiento)	Girar la vista horizontal y verticalmente (guiñada y cabeceo)
Esc	Cerrar el programa

2.4 Resumen general de controles

Elemento	**Tecla(s)**	**Efecto**
Cámara	W / A / S / D + ratón	Navegación libre por el rancho
Iluminación	Barra espaciadora	Enciende o apaga el color pulsante de la luz puntual central
Caballo	Flechas	Mueven al caballo sobre el terreno
Molino	M / N	Abren / cierran las dos puertas
Cochera	O / P	Suben / bajan la puerta
Tractor	U / I	Giran el volante a la izquierda / derecha

Elemento	Tecla(s)	Efecto
Tractor	T / Y	Avanzar / retroceder
Toro	X / Z	Caminar hacia adelante / retroceder
Toro	C / V	Girar a la izquierda / derecha
Keyframes	K / L / B	Guardar / reproducir o detener / borrar todos
Programa	Esc	Salir

2.5 Iluminación

La escena usa tres tipos de luz: una luz direccional (el sol) que ilumina todo el terreno, una luz tipo linterna (reflector azulado) que sale de la cámara y apunta hacia donde mira el usuario, y una luz puntual situada en (0, 2, 0). Al presionar la **barra espaciadora** se activa el color pulsante de la luz puntual; al presionarla otra vez se apaga. Con la luz apagada la escena conserva la iluminación del sol.

Nota de uso: conviene presionar y soltar la barra espaciadora con rapidez. Si se mantiene presionada, el repetidor del teclado puede alternar el estado varias veces y producir parpadeo.

2.6 Caballo

Las flechas del teclado desplazan al caballo: **flecha arriba** y **flecha abajo** lo mueven sobre el eje X (positivo y negativo, respectivamente); **flecha izquierda** y **flecha derecha** lo mueven sobre el eje Z. La velocidad es baja a propósito para permitir un posicionamiento fino.

2.7 Molino de viento

- Las aspas giran de manera continua por sí solas, simulando la acción del viento.
- **M:** abre las dos puertas de la base (se abren de forma simétrica, cada una sobre su bisagra) hasta un máximo de 120 grados.
- **N:** cierra las puertas hasta dejarlas en 0 grados.

2.8 Cochera

- **O:** sube la puerta de la cochera, con un tope de apertura de 2.5 unidades.
- **P:** baja la puerta hasta cerrarla por completo.

2.9 Tractor

El tractor se maneja como un vehículo real: el volante y el avance son controles independientes y la dirección del tractor solo cambia cuando éste está en movimiento.

- **U / I:** giran el volante a la izquierda o a la derecha, hasta un máximo de 35 grados hacia cada lado. Las ruedas delanteras se orientan visiblemente.
- **T:** avanza. **Y:** retrocede. Las cuatro ruedas ruedan mientras el tractor avanza.

- Para dar una curva: gire el volante (U o I) y después avance (T). Entre más cerrado esté el volante, más cerrada será la curva. Para enderezar, gire el volante en sentido contrario hasta que las ruedas queden rectas.
- El tractor inicia dentro de la cochera.

Nota: estas teclas se procesan con cada evento de teclado; si el movimiento se siente lento, mantenga presionada la tecla para aprovechar el autorrepetido del sistema.

2.10 Toro

El toro es un modelo articulado: la cabeza y las cuatro patas son piezas independientes unidas al cuerpo por pivotes. Al caminar mueve las patas en trote diagonal (la pata delantera izquierda y la trasera derecha avanzan juntas).

Tecla	Acción
X	Caminar hacia adelante (el frente del toro es hacia donde apunta la cabeza)
Z	Caminar hacia atrás (más lento que hacia adelante)
C	Girar a la izquierda (si está detenido, da pasos pequeños mientras gira)
V	Girar a la derecha

La cabeza se mueve sola en todo momento: mira lentamente a los lados y cabecea con suavidad. Al soltar las teclas, las patas regresan suavemente a su posición neutra.

2.11 Abeja

La abeja vuela en círculos continuos a 5 unidades de altura sobre el centro del rancho y no requiere control del usuario. Se dibuja con mezcla de color (blending) para que respete la transparencia de su textura.

2.12 Sistema de animación por keyframes

Este sistema permite grabar una secuencia de poses de toda la escena y reproducirla de manera automática. El teclado guarda el estado actual de los elementos animables como un fotograma clave, y al reproducir el programa interpola entre un fotograma y el siguiente.

Tecla	Función	Detalle
K	Guardar keyframe	Captura el valor actual de todas las variables animables. Máximo 30. No funciona durante la reproducción.
L	Reproducir / detener	Requiere al menos 2 keyframes. Reinicia la escena al primer fotograma y reproduce. Si ya está reproduciendo, la detiene.
B	Borrar todos	Elimina todos los keyframes guardados (solo si no se está reproduciendo).

Procedimiento para crear una animación

1. Coloque los elementos en su pose inicial con los controles manuales (caballo, tractor, toro, puertas, etc.) y presione **K**. La consola mostrará "KeyFrame 1 guardado".
2. Mueva los elementos a una nueva pose; por ejemplo, avance el tractor con T, abra la puerta de la cochera con O y camine con el toro con X. Presione **K** de nuevo.
3. Repita el paso anterior para cada pose intermedia o final que desee.
4. Presione **L** para reproducir. La escena vuelve a la primera pose y se mueve hacia cada keyframe en 3 segundos por tramo, con velocidad constante.
5. Para interrumpir la reproducción, presione **L** de nuevo. Para empezar una animación nueva, presione **B** y repita el procedimiento.

Qué se guarda y qué no

- **Se guarda:** posición del caballo, la vaca y el puerquito, aspas y puertas del molino, posición, rumbo, volante y llantas del tractor, puerta de la cochera, y posición, rumbo y fase de marcha del toro.
- **No se guarda:** la cámara (siempre queda libre), el vuelo de la abeja y el movimiento automático de la cabeza del toro (que continúa sola durante la reproducción).
- Durante la reproducción los controles manuales de las animaciones se desactivan; la cámara sigue funcionando, por lo que el usuario puede recorrer la escena mientras observa la animación.
- Las aspas del molino giran por sí solas cuando no se reproduce una animación; durante ella obedecen a los keyframes.
- Mensajes de la consola: "Reproduciendo N keyframes", "Termina animacion", "Animacion detenida", "Keyframes borrados" y "Necesitas al menos 2 keyframes (tecla K)".

Recomendación para la demostración: guarde el primer keyframe con la escena en reposo y registre, en cada keyframe siguiente, cambios claros en varios elementos a la vez (tractor, cochera y toro), de modo que la reproducción muestre las animaciones complejas y sencillas en una sola secuencia.

2.13 Solución de problemas

Síntoma	Causa probable y solución
No se ven modelos, con sus respectivos colores.	Descargue los archivos actualizados: lighting.frag, Mesh.h, Model.h
La luz puntual no se ve	Presione la barra espaciadora para activar su color pulsante.
Al presionar L aparece "Necesitas al menos 2 keyframes"	Guarde al menos dos keyframes con K antes de reproducir.
K no guarda nada	No se puede guardar durante la reproducción, o ya se llegó al máximo de 30 keyframes. Detenga con L o borre con B.
El tractor no gira al mover el volante	La dirección solo cambia con el tractor en movimiento. Gire el volante y avance con T o Y.

3 Manual técnico y documentación del proyecto

3.1 Objetivos

Objetivo general

Diseñar e implementar un entorno virtual tridimensional interactivo de un rancho, en C++ con OpenGL, que aplique los conocimientos de computación gráfica del curso: modelado, texturizado, iluminación, transformaciones jerárquicas y animación generada por código.

Objetivos específicos

- Integrar los modelos al programa mediante la clase `Model`, basada en Assimp, y colocarlos en la escena con matrices de modelo.
- Implementar un sistema de iluminación con luz direccional, luz puntual y luz de reflector mediante shaders GLSL.
- Programar una cámara libre en primera persona controlada con teclado y ratón, independiente de la velocidad del equipo mediante deltaTime.
- Construir animaciones jerárquicas con relaciones padre-hijo entre matrices (tractor, molino, cochera y toro).
- Implementar un sistema de keyframes con interpolación lineal que grabe y reproduzca el estado de todas las variables animables.
- Cumplir con cinco animaciones coherentes con el contexto del rancho: tres sencillas y dos complejas.

3.2 Alcance del proyecto

La simulación incluye, en tiempo real:

- Un rancho tridimensional de 90 x 90 unidades delimitado por una cerca perimetral de madera, con terreno texturizado y cielo de color.
- Doce elementos de escena (descritos en la sección 3.7), de los cuales algunos fueron modelados en Blender y cargados con Assimp.
- Navegación libre mediante cámara en primera persona con teclado y ratón.
- Tres fuentes de luz activas (direccional, puntual y reflector) con color pulsante controlable por el usuario.
- Tres animaciones sencillas (aspas del molino, puerta de la cochera, movimiento del caballo y movimiento de la abeja) y dos complejas (tractor y toro).
- Un sistema de keyframes que permite grabar hasta 30 poses y reproducirlas con interpolación lineal.

Queda fuera del alcance: detección de colisiones, física, sonido, ciclo día-noche, sombras proyectadas, inteligencia artificial de los animales y animación esquelética de los modelos.

3.3 Limitantes

- **Rendimiento y carga de dibujo.** Cada pieza se dibuja con su propia llamada de dibujo y no hay instanciación; la cerca perimetral, por ejemplo, requiere 184 llamadas por cuadro (46 tramos por lado). Esto es suficiente para el equipo de pruebas, pero crecería mal con una escena mayor.
- **Luces.** El shader soporta varias luces puntuales, pero en esta versión solo la luz puntual 0 está configurada; las otras tres posiciones del arreglo permanecen en el origen y se muestran como cubos de depuración sin aportar iluminación.
- **Interpolación.** Los keyframes se interpolan linealmente (con opción de suavizado cúbico desactivada por defecto). El movimiento entre poses tiene velocidad constante y no sigue curvas; no hay trayectorias curvas, ni interpolación de rotaciones mediante cuaterniones.
- **Ángulos.** Los giros se interpolan como números reales independientes (grados), por lo que no se toma el camino más corto entre ángulos; conviene que los giros entre keyframes sean menores a 180 grados.
- **Capacidad de keyframes.** Máximo de 30 fotogramas almacenados en memoria; no se guardan en disco, por lo que se pierden al cerrar el programa.
- **Entrada de teclado.** Los controles del molino, el tractor y la cochera se evalúan dentro del callback de teclado, por lo que su velocidad depende del autorrepetido del sistema operativo y no de deltaTime.
- **Sin colisiones.** El usuario, el tractor y el toro pueden atravesar edificios y cercas.
- **Hardware.** El programa usa una ventana fija de 2024 x 1080 y requiere una tarjeta gráfica con OpenGL 3.3 o superior. El tamaño de la ventana no es redimensionable en tiempo de ejecución.

3.4 Metodología de software aplicada

Se utilizó un enfoque **incremental e iterativo**, con integración continua de cada elemento a la escena y revisión visual inmediata. Cada iteración añadió un elemento o una capacidad nueva (modelo, jerarquía, luz, animación) y terminó con una prueba funcional en ejecución.

La arquitectura es **modular y orientada a objetos** en los componentes de infraestructura:

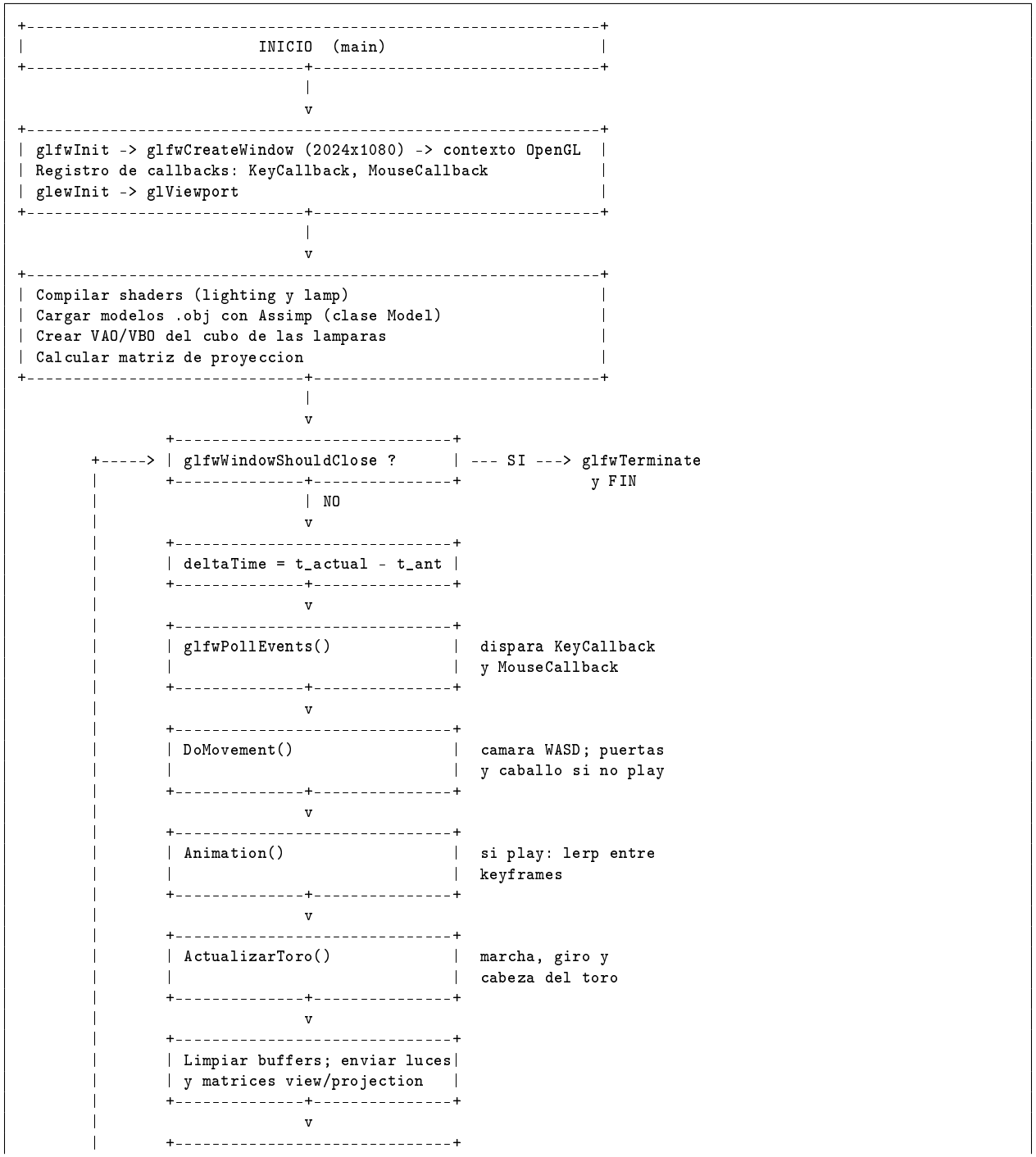
Componente	Rol en la arquitectura
GLFW	Creación de la ventana, contexto OpenGL, ciclo de eventos, teclado, ratón y reloj (glfwGetTime).
GLEW	Carga de las extensiones y funciones modernas de OpenGL (glewExperimental = GL_TRUE).
GLM	Matemática de gráficas: vectores, matrices de modelo, vista y proyección, y funciones de transformación.
Assimp (clase Model)	Importación de los archivos OBJ con sus mallas y materiales; Model.Draw(shader) los dibuja.
SOIL2 / stb_image	Carga de imágenes para las texturas.
Clase Shader	Compila y enlaza los programas GLSL a partir de los archivos .vs y .frag; expone Use() y Program.
Clase Camera	Mantiene posición, orientación, matriz de vista y procesa el teclado y el ratón.

La lógica de la aplicación se dividió en funciones con responsabilidad única (DoMovement, Animation, ActualizarToro, KeyCallback, MouseCallback, MoverTractor, PiezaToro, RotX y las funciones del sistema de

keyframes), y el estado de cada elemento animable se concentra en variables globales con nombres descriptivos, lo que permite que el sistema de keyframes las grabe todas de forma uniforme.

3.5 Diagrama de flujo del software

El programa sigue el esquema clásico de un ciclo de renderizado en tiempo real: inicialización una sola vez y, después, un ciclo que atiende eventos, actualiza el estado y dibuja un cuadro por iteración.



```
        |       | DIBUJO: piso, establo,       |
        |       | animales, abeja (blend),     |
        |       | molino, tractor, cochera,    |
        |       | toro, cercas, lamparas       |
        |       +--------------+---------------+
        |                      v
        |       +------------------------------+
        |       | glfwSwapBuffers()            |
        |       +--------------+---------------+
        |                      |
        +----------------------+
```

Eventos asíncronos: durante glfwPollEvents se invoca KeyCallback, que atiende las teclas de keyframes (K, L, B), los controles de molino, tractor y cochera, la tecla Esc y la tecla de luz. MouseCallback se invoca con cada movimiento del ratón y actualiza la orientación de la cámara.

Flujo del sistema de keyframes

```
Tecla K  --> (play = falso?) --> saveFrame(): copia varsAnim[] a
                                 KeyFrame[FrameIndex]; FrameIndex++
Tecla B  --> (play = falso?) --> FrameIndex = 0
Tecla L  --> play activo     --> detener (play = falso)
         --> FrameIndex > 1  --> resetElements(): aplicarFrame(0)
                                 play = verdadero; playIndex = 0
                                 tTramo = 0
         --> en otro caso    --> mensaje "Necesitas al menos 2..."

Cada cuadro, Animation():
   tTramo += deltaTime / DURACION_TRAMO
   si tTramo >= 1:  playIndex++; aplicarFrame(playIndex); tTramo = 0
                    si era el ultimo tramo -> play = falso (termina)
   en otro caso:    var = a + (b - a) * e     (para todas las variables)
```

3.6 Diagrama de Gantt

Cronograma de desarrollo por fases, expresado en semanas de trabajo. Las celdas oscuras indican el periodo de actividad de cada fase.

Fase	**S1** 21/09/2026	**S2** 28/09/2026	**S3** 05/10/2026	**S4** 12/10/2026	**S5** 19/10/2026	**S6** 26/10/2026	**S7** 02/11/2026
1. Planeación y referencias	■	■					
2. Modelado en Blender		■	■				
3. Exportación OBJ e integración con Assimp		■	■	■			
4. Escena base, iluminación y texturas				■	■	■	
5. Jerarquías de matrices (molino, tractor, cochera, toro)				■	■	■	■
6. Sistema de keyframes						■	■
7. Depuración y pruebas							■
8. Documentación y video							■

Fase	Actividades principales
1. Planeación y referencias	Elección del tema (rancho), recopilación de imágenes de referencia, lista de objetos y reparto de tareas.
2. Modelado en Blender	Modelado de tractor, molino, pozo, toro; materiales de color y texturas.
3. Exportación e integración	Exportación a OBJ separando las partes móviles; carga con la clase Model (Assimp) y colocación inicial en la escena.
4. Escena, luces y texturas	Piso texturizado, shaders de iluminación (direccional, puntual y reflector), blending de la abeja y cerca perimetral.
5. Jerarquías de matrices	Molino (aspas y puertas), tractor con rumbo y dirección, puerta de la cochera y toro con pivotes en cuello y patas.
6. Sistema de keyframes	Lista de variables animables, guardado, reproducción por tramos, interpolación y reinicio de la escena.
7. Depuración y pruebas	Ajuste de pivotes y escalas, corrección de la marcha del toro y del giro del tractor, pruebas de reproducción.
8. Documentación y video	Manual de usuario, manual técnico, video explicativo y preparación de la exposición.

3.7 Inventario de objetos de la escena

Todos los elementos que provienen de archivos OBJ se cargan con la clase `Model` (Assimp) y se dibujan con `Model.Draw(shader)`. La siguiente tabla documenta formalmente los doce elementos de la escena.

N.º	Elemento	Archivos / origen	Comportamiento
1	Establo (casita roja)	RanchoNuevo2.obj, vallas.obj, PuertasCentralesDentro.obj	Estructura estática (traslación -0.3, -3.8, -6.8; escala 5.3), color por material. Puertas interiores deslizantes con 3 / 4.
2	Caballo	caballo.obj	Dentro del establo; se desplaza con las flechas o por keyframes.
3	Vaca	vaca.obj	Dentro del establo; posición controlada por keyframes.
4	Cochera (casita azul)	Cochera_Estructura.obj, Cochera_Puerta.obj	Posición (-12, 0, 5). Puerta que sube y baja con O / P (animación sencilla).
5	Tractor	Tractor_Cuerpo.obj y cuatro ruedas independientes (Tras_Izq, Tras_Der, Del_Izq, Del_Der)	Animación compleja: chasis con rumbo, ruedas rodando, ruedas delanteras con dirección.
6	Abeja	abeja.obj	Vuelo circular continuo a 5 unidades de altura, con blending (transparencia).
7	Molino de viento	Molino_Base.obj, MolinoAspas.obj, Molino_Puerta_Izq.obj, Molino_Puerta_Der.obj	Posición (15, 0, 1). Aspas con rotación continua y puertas con bisagra (animaciones sencillas).
8	Pozo	Elemento estático de ambientación	Sin animación; se aprecia junto al establo.
9	Puerquito	cerdo.obj	Posición controlada por keyframes.
10	Toro	Toro_Cuerpo.obj, Toro_Cabeza.obj y cuatro patas (DelIzq, DelDer, TrasIzq, TrasDer)	Animación compleja: jerarquía de seis piezas sobre pivotes; marcha en trote diagonal y cabeza automática.

N.º	Elemento	Archivos / origen	Comportamiento
11	Lámparas	Cubo generado por código (VAO/VBO) con lamp.vs / lamp.frag	Cuatro cubos de 0.2 de escala en las posiciones de las luces puntuales; la luz 0 (0, 2, 0) ilumina con color pulsante (espacio).
12	Cercas de madera	Cerca_Tramo.obj	Un tramo de 2 unidades repetido 184 veces para formar el perímetro de 90 x 90.

Además, el terreno (piso.obj) se escala a 50 x 1 x 50 y se dibuja con texturas.

3.8 Documentación del código

Estructura general del archivo principal

Todo el programa se encuentra en un único archivo fuente, organizado en las siguientes secciones, de arriba hacia abajo:

1. **Inclusión de bibliotecas:** iostream, cstdio, cmath, GLEW, GLFW, stb_image, GLM, SOIL2 y las clases propias Shader, Camera y Model.
2. **Prototipos y constantes de ventana:** KeyCallback, MouseCallback, DoMovement, Animation, ActualizarToro; ventana de 2024 x 1080.
3. **Estado global:** cámara, arreglo keys[1024], luces, variables de cada animación y datos del sistema de keyframes.
4. **Funciones auxiliares:** saveFrame, aplicarFrame, idxVar, resetElements, PiezaToro, RotX, MoverTractor.
5. **main():** inicialización, carga de modelos, ciclo de renderizado y dibujo.
6. **Funciones de actualización y eventos:** DoMovement, ActualizarToro, KeyCallback, Animation y MouseCallback.

Gestión de shaders e iluminación

Se usan dos programas de shaders. **lighting** (lighting.vs y lighting.frag) dibuja todos los modelos con iluminación de Phong y **lamp** (lamp.vs y lamp.frag) dibuja los cubos de las lámparas sin iluminación. El programa lighting recibe, por cuadro, los siguientes uniformes:

Uniforme	Valor y función
viewPos	Posición de la cámara; se usa para el brillo especular.
dirLight	Luz direccional (el sol): dirección (-0.2, -1, -0.3), ambiental y difusa 0.6, especular 0.3.
pointLights[0]	Luz puntual en (0, 2, 0); color ambiental y difuso = |sin(t * Light1)| por componente (pulsante); atenuación constante 1.0, lineal 0.045, cuadrática 0.075.
spotLight	Reflector colocado en la cámara y dirigido por camera.GetFront(); color azulado (0.2, 0.2, 0.8); cutOff cos(12 grados) y outerCutOff cos(18 grados).
material.shininess	Brillo especular, 5.0.

Uniforme	Valor y función
model / view / projection	Matrices de transformación. view y projection se envían una vez por cuadro; model se envía antes de cada Draw.
uUseColorOnly	Verdadero para modelos que usan solo el color del material (establo, toro); falso para modelos con textura (piso, cerca).
transparency	Activa el manejo de transparencia en el fragment shader; se enciende solo al dibujar la abeja.

La tecla espaciadora alterna la variable `Light1` entre (0, 0, 0) y (0.2, 0.8, 1.0). Como el color de la luz puntual depende de abs(sin(t * Light1)), con Light1 = 0 la luz queda apagada y con valores distintos de cero pulsa con frecuencias diferentes en cada canal de color.

Transparencia de la abeja. Se habilita el blending con glEnable(GL_BLEND) y glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA), se activa el uniforme transparency, se dibuja el modelo y se desactivan ambos para no afectar el resto de la escena.

Clases Shader, Model y Camera

Clase	Responsabilidad	Uso en el proyecto
Shader	Lee, compila y enlaza el vertex y fragment shader; guarda el identificador del programa en Program.	lightingShader y lampShader; Use() activa el programa; Program permite buscar uniformes.
Model	Con Assimp lee el OBJ, construye mallas y materiales, y carga texturas (SOIL2). Draw(shader) dibuja todas las mallas.	Se crea un objeto Model por cada pieza (por ejemplo, cada rueda del tractor y cada pata del toro).
Camera	Guarda posición, ángulos (guiñada y cabeceo) y vectores de orientación; entrega GetViewMatrix, GetPosition, GetFront y GetZoom; ProcessKeyboard(dirección, deltaTime) y ProcessMouseMovement(dx, dy).	Cámara libre, inicia en (0, 2, 10). También posiciona y orienta el reflector.

Para evitar que la velocidad de la cámara dependa de los cuadros por segundo, ProcessKeyboard recibe `deltaTime`, que se calcula en cada iteración como la diferencia entre el tiempo actual (glfwGetTime) y el del cuadro anterior.

Transformaciones jerárquicas (matrices padre-hijo)

Las animaciones compuestas se resuelven construyendo una **matriz base (padre)** que posiciona y orienta el objeto completo, y derivando de ella la matriz de cada pieza (hijo) con transformaciones locales. En OpenGL con GLM, las transformaciones se aplican de derecha a izquierda; por eso el orden de escritura es: trasladar a la articulación, rotar y, al final, dibujar la pieza.

$$M_{hijo} = M_{padre} \cdot T(\text{pivote}) \cdot R(\theta)$$

Molino. Todas las piezas parten de `posMolino` = (15, 0, 1). La base se dibuja sin giro. Las aspas se trasladan a (0, 4.4, 2.3) y giran sobre el eje Z con `rotAspas`. Cada puerta se traslada a su bisagra, en (-0.5, 0, 1.82)

y (0.5, 0, 1.82), y gira sobre Y con `rotPuertaMolino` (la derecha con signo negativo, para abrir de manera simétrica).

Cochera. La estructura se coloca en `posCochera` = (-12, 0, 5). La puerta usa la misma posición más una traslación (0, alturaPuertaCochera, 3) y sube sin rotar.

Tractor. La matriz padre `tractorBase` se construye así:

$$M_{base} = T(x_t, 0, z_t) \cdot R_y(\psi) \cdot T(0, 0, -1.2)$$

donde (xt, zt) = (`tractorPosX`, `tractorPosZ`) es la posición del eje trasero y psi = `anguloTractor` es el rumbo. La última traslación (constante `TRACTOR_EJE_TRAS_Z` = 1.2) corre el modelo hacia atrás para que el chasis gire alrededor del eje trasero, como en un vehículo real. Las ruedas se derivan de `tractorBase`:

- Traseras: traslación a (±0.9, 0.7, 1.2) y rotación sobre X con `rotLlantasTractor` (rodamiento).
- Delanteras: traslación a (±0.7, 0.4, -1.3), rotación sobre Y con `anguloGiroDireccion` (dirección) y después sobre X con `rotLlantasTractor` (rodamiento). El orden dirección-rodamiento hace que la rueda gire sobre su eje ya orientada.

La función `MoverTractor(ds)` actualiza el estado con cada paso (negativo = adelante, positivo = reversa):

$$x_t \mathrel{+}= \sin(\psi)\, ds, \quad z_t \mathrel{+}= \cos(\psi)\, ds, \quad \psi \mathrel{-}= 0.15\, \delta\, ds, \quad \phi_{rueda} \mathrel{+}= 100\, ds$$

donde delta = `anguloGiroDireccion` (el volante, de -35 a 35 grados). Así la posición se acumula paso a paso en la dirección del rumbo (seno y coseno), el rumbo cambia en proporción al ángulo del volante (y solo al avanzar), y las ruedas giran 10 grados por cada 0.1 unidades recorridas.

El toro: jerarquía de seis piezas y pivotes

El toro consta de seis modelos: cuerpo, cabeza y cuatro patas. Cada pieza móvil se exportó con sus vértices en **coordenadas locales**, centrada en su propia articulación (cuello o cadera/hombro). Por esa razón se coloca con T(pivote) * R, sin la traslación inversa T(-pivote) que se necesitaría si los vértices estuvieran en coordenadas de mundo. La constante `TORO_OBJ_LOCAL = true` selecciona este modo y la función `PiezaToro(base, origen, rot)` lo implementa:

$$M_{pieza} = M_{base} \cdot T(\text{origen}) \cdot R\ (\cdot\, T(-\text{origen}) \text{ si } TORO_OBJ_LOCAL = \text{falso})$$

Pieza	Pivote (x, y, z)	Rotación aplicada
Cuerpo	(0, 0, 0): origen del toro	Ninguna (usa la matriz base)
Cabeza	PIVOTE_CUELLO (0.00, 1.40, -0.95)	Ry(toroCabezaYaw) * Rx(toroCabezaPitch)
Pata delantera izquierda	PIVOTE_DEL_IZQ (-0.42, 0.95, -0.55)	Rx(+angDel)
Pata delantera derecha	PIVOTE_DEL_DER (0.42, 0.95, -0.55)	Rx(-angDel)
Pata trasera izquierda	PIVOTE_TRAS_IZQ (-0.40, 0.95, 0.75)	Rx(-angTras)
Pata trasera derecha	PIVOTE_TRAS_DER (0.40, 0.95, 0.75)	Rx(+angTras)

La matriz base `toroBase` combina la posición (toroPosX, -hundimiento, toroPosZ), el rumbo `toroYaw` sobre Y y la escala `TORO_ESCALA`. El frente del toro es el eje -Z local.

Marcha en trote diagonal. Con la fase de marcha phi = `toroFase` y el factor de marcha m = `toroMarcha` (0 parado, 1 caminando), el ángulo de balanceo es:

$$\alpha_{del} = 24^\circ \cdot \sin(\varphi) \cdot m \qquad \alpha_{tras} = 20^\circ \cdot \sin(\varphi) \cdot m$$

Los signos opuestos en la tabla anterior hacen que la pata delantera izquierda y la trasera derecha avancen juntas mientras sus opuestas retroceden. La fase no depende del tiempo sino de la distancia recorrida (`toroFase += dist * FASE_POR_METRO`, con 4 rad por metro), de modo que las patas no patinan sobre el suelo y, si el toro retrocede, el ciclo se invierte solo.

Hundimiento del cuerpo. Cuando las patas se inclinan, su altura efectiva disminuye. Para que la pata más inclinada siga tocando el piso, el cuerpo baja:

$$h = 0.95\,(1 - \cos\alpha_{max}), \quad \alpha_{max} = \max(|\alpha_{del}|, |\alpha_{tras}|)$$

Suavizado de la marcha. El factor `toroMarcha` tiende al objetivo (1 caminando, 0 detenido) con un filtro exponencial: toroMarcha += (objetivo - toroMarcha) * min(1, 8 * deltaTime). Esto hace que las patas arranquen y se detengan de manera gradual.

Cabeza automática. En cada cuadro, `ActualizarToro` calcula con senos de distinto periodo el giro lateral (hasta aproximadamente 22 grados) y el cabeceo de la cabeza, más un ligero cabeceo sincronizado con la marcha. Si se pusiera `CABEZA_POR_KEYFRAMES = true`, durante la reproducción la cabeza obedecería a los keyframes.

Control del toro. X avanza con velocidad `VEL` = 1.2 unidades/s, Z retrocede al 60 por ciento, C y V giran a 70 grados/s. Si solo se gira, el toro avanza al 40 por ciento de VEL para dar pasos en lugar de patinar. La posición se actualiza en la dirección del frente: toroPosX += -sin(yaw) * dist; toroPosZ += -cos(yaw) * dist.

Sistema de interpolación de fotogramas clave

El sistema se basa en una lista de apuntadores a todas las variables animables, `varsAnim[]`, cuyo tamaño `N_VARS` se calcula automáticamente. Cada keyframe es una estructura `FRAME` con un arreglo de N_VARS valores. De este modo, añadir una variable animable solo requiere agregar su dirección a la lista.

Elemento	Descripción
varsAnim[] / N_VARS	Variables animables: puertas, posiciones de caballo, vaca y puerquito, molino (aspas y puertas), tractor (avance, llantas, volante, rumbo, x, z), cochera, y toro (x, z, rumbo, fase, cabeza yaw y pitch).
FRAME y KeyFrame[MAX_FRAMES]	Almacén de hasta 30 fotogramas; FrameIndex indica cuántos hay.
saveFrame()	Copia el valor actual de cada variable al siguiente fotograma libre (tecla K).
aplicarFrame(k)	Escribe en las variables animables los valores del fotograma k.
resetElements()	Aplica el fotograma 0 y apaga la marcha del toro antes de reproducir.

Elemento	Descripción
idxVar(p)	Devuelve la posición de una variable en la lista; permite leer su valor dentro de un keyframe (se usa en ActualizarToro).
play, playIndex, tTramo	Estado de reproducción: bandera, tramo actual (de KeyFrame[playIndex] a KeyFrame[playIndex + 1]) y avance dentro del tramo (0 a 1).
DURACION_TRAMO	3 segundos por tramo; la animación depende del tiempo (deltaTime), no de los cuadros por segundo.
SUAVIZAR_KEYFRAMES	false (valor actual): interpolación lineal pura. true: suavizado cúbico smoothstep.

En cada cuadro, `Animation()` avanza `tTramo` en deltaTime / DURACION_TRAMO y calcula todas las variables entre el fotograma de salida a y el de llegada b:

$$v = a + (b-a)\cdot e, \qquad e = t \ \ (\text{lineal}) \quad \text{o} \quad e = t^2(3-2t) \ \ (\text{suavizado})$$

Al llegar a t = 1, el programa fija las variables exactamente en los valores del siguiente fotograma, pasa al siguiente tramo y, si era el último, termina la reproducción. Como las posiciones, el rumbo, el volante y las llantas se interpolan juntos, el tractor y el toro recorren trayectorias coherentes: avanzan, giran y ruedan o caminan al mismo tiempo.

Durante la reproducción `ActualizarToro` no lee el teclado; solo decide si las patas deben caminar en el tramo actual comparando el cambio de fase y de posición entre los dos keyframes, y enciende o apaga la marcha de forma suave.

Animaciones del proyecto

Tipo	Animación	Descripción técnica
Sencilla	Aspas del molino	Rotación continua sobre el eje Z a 0.5 grados por cuadro (si no se reproducen keyframes).
Sencilla	Puertas del molino	Rotación sobre bisagras (eje Y) entre 0 y 120 grados, simétrica.
Sencilla	Puerta de la cochera	Traslación vertical con límites entre 0 y 2.5 unidades.
Sencilla	Vuelo de la abeja	Giro circular continuo sobre Y a 5 unidades de altura.
Compleja	Tractor	Rumbo acumulado con seno y coseno, giro alrededor del eje trasero, dirección proporcional al volante, rodamiento de cuatro ruedas.
Compleja	Toro	Jerarquía de seis piezas con pivotes, trote diagonal sincronizado con la distancia, hundimiento del cuerpo y cabeza automática.

Cerca perimetral

La cerca se genera con cuatro ciclos `for` que repiten el modelo `Cerca_Tramo` cada 2.0 unidades, entre -45 y 45 en X y en Z (46 tramos por lado, 184 en total). Los lados trasero e izquierdo y derecho se giran 180, 90 y -90 grados sobre Y para que el frente de cada tramo mire hacia afuera. Se dibuja con uUseColorOnly en falso para conservar sus texturas.

Resumen de funciones

Función	Descripción
main()	Inicializa ventana y OpenGL, carga shaders y modelos, y ejecuta el ciclo de renderizado y dibujo.
DoMovement()	Aplica las teclas mantenidas: puertas (1 a 4) y caballo (flechas) si no se reproduce, y la cámara (WASD).
Animation()	Reproduce los keyframes: avanza el tramo e interpola todas las variables.
ActualizarToro()	Mueve y orienta al toro, calcula la fase de marcha, suaviza las patas y anima la cabeza.
KeyCallback()	Atiende K, L, B, controles del molino, tractor y cochera, Esc y espacio; mantiene el arreglo keys[].
MouseCallback()	Calcula el desplazamiento del ratón y lo envía a la cámara.
MoverTractor(ds)	Avanza o retrocede el tractor y actualiza posición, rumbo y llantas.
PiezaToro(base, origen, rot)	Matriz de una pieza articulada del toro.
RotX(grados)	Matriz de rotación sobre X, usada como péndulo de las patas.
saveFrame, aplicarFrame, idxVar, resetElements	Operaciones del sistema de keyframes.

Observaciones y mejoras futuras

- Configurar las luces puntuales restantes (posiciones 1 a 3) o retirar los cubos que no iluminan.
- Mover los controles de molino, tractor y cochera de KeyCallback a DoMovement para que dependan de deltaTime y no del autorrepetido del teclado.
- Agregar detección de colisiones y sombras proyectadas.
- Guardar y cargar los keyframes en un archivo para conservarlos entre ejecuciones.
- Usar instanciación para dibujar la cerca con una sola llamada.

4 Conclusiones

4.1 Nequis Sarabia Jorge Luis

Cuando empezamos este proyecto pensé que lo más difícil sería el modelado. Con el tiempo me di cuenta de que lo realmente retador era hacer que todo se comportara como un conjunto: que una rueda gire sobre su propio eje mientras el tractor avanza, que la puerta se abra desde su bisagra y no desde el centro, que una pata se mueva como un péndulo sin separarse del cuerpo. Entender el orden de las transformaciones, trasladar, rotar y dibujar, y por qué una matriz hija se construye a partir de la de su padre, fue de esos momentos en los que algo que parecía abstracto en clase de pronto se vio en pantalla.

Los errores nos enseñaron mucho. Cuando una pieza del toro aparecía girando alrededor del centro del mundo y no de su articulación, comprendí la diferencia entre exportar los vértices en coordenadas locales y en coordenadas de mundo, y cómo cambia la multiplicación de matrices en cada caso. Algo parecido ocurrió con el tractor: hacerlo girar alrededor de su eje trasero, y no de su centro, hizo que la curva se sintiera real. Son detalles pequeños que marcan la diferencia entre una animación que se ve bien y una que se ve rara.

También valoro mucho el sistema de keyframes. Ver que una lista de apuntadores y una interpolación lineal bastan para grabar y repetir una escena completa me mostró que las herramientas más poderosas muchas veces son simples.

4.2 Jiménez Vázquez Sergio Yahir

Este proyecto me hizo ver la computación gráfica de otra manera. Antes veía un videojuego o una animación y pensaba en el resultado final; ahora pienso en las matrices, en el ciclo de renderizado y en todo lo que ocurre en cada cuadro. Pasar de un modelo en Blender a verlo caminar, girar o abrirse dentro de nuestro propio programa fue muy satisfactorio, y me enseñó que el modelado y la programación son dos mitades de un mismo trabajo: un buen modelo mal exportado no se puede animar, y una buena animación necesita un modelo pensado para ella.

Lo que más disfruté fue resolver el toro, porque exigía pensar como un animador y como un programador al mismo tiempo. Había que decidir dónde estaban los pivotes, cómo se coordinan las patas en un trote diagonal, y cómo evitar que los pies patinen o atraviesen el suelo. Descubrir que bastaba ligar la fase de la marcha a la distancia recorrida, y no al tiempo, fue una de esas ideas que, cuando funcionan, se sienten casi como magia. Aprendí también a depurar con paciencia: cambiar un número, volver a ejecutar y observar.

Me llevo, además, el aprendizaje de trabajar en equipo: dividir el trabajo, probar cada cambio y avanzar poco a poco.

5 Referencias

- de Vries, J. LearnOpenGL. https://learnopengl.com
- The Khronos Group. OpenGL Documentation. https://www.khronos.org/opengl/
- GLFW. Documentation. https://www.glfw.org/documentation.html
- GLM. OpenGL Mathematics. https://github.com/g-truc/glm
- The Open Asset Import Library (Assimp). https://www.assimp.org
- GLEW. The OpenGL Extension Wrangler Library. https://glew.sourceforge.net
- Blender Foundation. Blender Manual. https://docs.blender.org/manual/
- Espinoza Urzúa, E. Material del curso y código base de Laboratorio de Computación Gráfica e Interacción Humano-Computadora. Facultad de Ingeniería, UNAM.